Informatique et Jeux

Emilie Morvant

Faculté des Sciences et Techniques
Université Jean Monnet de Saint-Etienne

Licence 1 - Semestre 1
2016 - 2017

Qui suis-je ?

- Maître de Conférence
  à l'Université Jean Monnet, St-Etienne, France
  au Laboratoire Hubert Curien
  dans l'équipe d'apprentissage automatique
  responsable de la Licence 2 d'informatique
- Domaine de recherche :
  - Domaine principal : Apprentissage Automatique *(Machine Learning)*
  - Théorie de l'apprentissage automatique statistique
  - Applications à des pbs de vision par ordinateur *(Computer Vision)*
- Comment me joindre :
  Mail : `emilie.morvant@univ-st-etienne.fr`
  Twitter : @EmilieMorvant

Disclaimer

Vous allez voir des vidéos commerciales

- Je ne fais partie d'aucune compagnie privée
- Je n'ai pas d'action dans les compagnies concernées par les vidéos
- Mon objectif est de vous montrer des exemples "parlants" récents

Ce dont on va parler

- Casse tête (Sudoku, picross, . . .)
  ↪ Problème de satisfaction de contraintes
- Jeux de stratégies classiques (Echecs, Dames, Othello, Go, . . .)
  ↪ IA et MinMax (*c.f.* TD sur les arbres)
  ↪ Jeux de Nim et stratégies gagnantes
- Jeux Vidéos
  ↪ Comment fait-on ?
  ↪ Infographie
  ↪ À vous de jouer...

À vous de jouer...

Je vous propose de rendre intéractif ce cours. . .

Twitter : #L1STINFO16

Déroulement :

- 2 premières séances "classiques" : mais vous pouvez poser vos questions via le hahstag
- 3ème séance le 4/11 : C'est vous qui décidez, jusqu'au **25 octobre** !
  ↪ Postez sur #L1STINFO15, ce dont vous souhaiteriez parler

Vous n'utilisez pas twitter ?
Pas de problème envoyez moi un petit mail
`emilie.morvant@univ-st-etienne.fr`

Une structure de données utile
Les graphes

Les graphes

Structure de données

En informatique, une structure de données est une structure logique destinée à contenir des données, afin de leur donner une organisation permettant de **simplifier leur traitement**

Exemples :
- Singletons (constantes, variables, ensembles, ...)
- Tableaux (et matrices = tableaux à deux dimensions)
- Listes (*c.f.* partie sur les machines abstraites)
- Arbres (*c.f.* partie sur les arbres)
- Graphes (une généralisation des arbres)

Problème

Comment manipuler les données dans ces structures ?
i.e. Comment comprendre leur structure logique ?

Les graphes

Quelques définitions sur les graphes

Graphe non orienté

Un graphe **non orienté** $G = (X, A)$ est un couple de deux ensembles:
- $X = \{x_1, x_2, \ldots, x_n\}$ dont les éléments sont les **sommets** du graphe
- $A = \{a_1, a_2, \ldots, a_n\}$ dont les éléments sont les **arrêtes** du graphe

Soit $a = (x, y) \in A$ (avec $x \in X$ et $y \in X$), on dit que a est l'arête de G d'**extrémité** x et y ou que a **joint** x et y ou que a **passe par** x et y
Les sommets x et y sont dits **adjacents** dans G

Rq : A est une partie de l'ensemble $P^2(X)$ des parties à deux éléments de X

Graphe orienté

Un graphe est **orienté** si les arêtes ont un sens. Elles sont alors appelées **arcs**. Par exemple $(x, y) \in A$ indique qu'il y a un arc d'**origine** x et d'**extrémité finale** y

Les graphes

Quelques définitions sur les graphes

Successeurs et prédécesseurs d'un sommet

Soit $G = (X, A)$ un **graphe orienté**

L'ensemble $\Gamma^+(x)$ des sommets **successeurs** d'un sommet $x \in X$ est l'ensemble des sommets y tels que $(x, y) \in A$

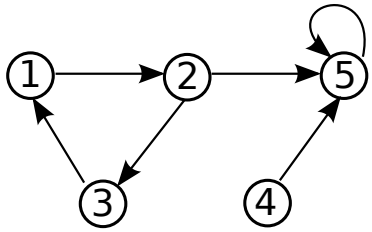
L'ensemble $\Gamma^-(x)$ des sommets **prédécesseurs** d'un sommet $x \in X$ est l'ensemble des sommets w tels que $(w, x) \in A$

Circuit

Dans un graphe orienté, un circuit est une suite d'arêtes consécutives (chemin) dont les deux sommets extrémités sont identiques (= boucle)

Les graphes

Exercice : On considère le graphe ci-dessous



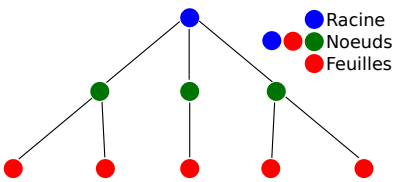
- Définir le graphe $G = (X, A)$
- $\Gamma^+(2) = ?$
- $\Gamma^-(5) = ?$
- $\Gamma^-(4) = ?$
- Donnez un circuit du graphe

Les arbres

Arbre

Un **arbre** est un graphe particulier qui contient une **racine**, des **nœuds** et des **feuilles**.

C'est un graphe non orienté, acyclique et connexe.

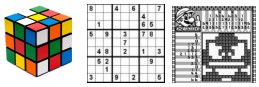


Problème de satisfaction de contraintes (CSP)

Problème de satisfaction de contraintes

Motivation : Les casse-têtes

Problèmes à un seul joueur avec :
- une situation de départ
- un objectif final



$$\begin{array}{r} \text{SEND} \\ +\text{MORE} \\ \hline \text{MONEY} \end{array}$$

Méthode de résolution : énumérer des solutions possibles

tant que on n'a pas trouvé de solution **ET** qu'on n'a pas exploré tous les choix **faire**
  on choisit une solution possible
  ⇔ qui vérifie toutes les **contraintes** spécifiées dans les règles du jeu
  si l'on n'arrive pas à une contradiction
  ⇔ les contraintes sont toujours vérifiée **alors**
    on a trouvé une solution
    ⇔ le problème a au moins une solution qui est la courante
  fin si
fin tant que
le problème n'a pas de solution
⇔ on a exploré tous les choix sans trouver de solution

Problème de satisfaction de contraintes

Que peut faire l'informatique ?

- Trouver un langage pour lequel il est facile de poser/définir des contraintes

⇒ le langage doit doit ressembler au langage naturel

Modéliser les contraintes

- Trouver une méthode de résolution des problèmes qui mimique la technique humaine

Automatiser la "réflexion"

Problème de satisfaction de contraintes

Qu'est ce qu'une contrainte ?

Contrainte

Une contrainte est une **relation logique** (une propriété qui doit être vérifiée) entre différentes inconnues, appelées **variables**, chacune prenant ses valeurs dans un ensemble donné appelé **domaine**.

Une contrainte restreint les valeurs que peuvent prendre simultanément les variables.

Exemple : $x + 3 \times y = 12$
Elle restreint les valeurs que l'on peut affecter en même temps aux variables x et y

Problème de satisfaction de contraintes

Problème de satisfaction de contraintes

Un CSP *(Constraint Satisfaction Problem)* est un problème modélisé sous la forme d'un **ensemble de contraintes** (3) posées sur des **variables** (1), chacune des variables prenant ses valeurs dans un **domaine** (2).

La définition formelle d'un CSP est un triplet (X, D, C) tel que :

(1) $X = \{X_1, X_2, \ldots, X_n\}$ est l'ensemble des **variables** (inconnues) du problème

(2) $D : X \to D(X)$ est la fonction qui associe à chaque variable $X_i \in X$ son **domaine** $D(X_i)$, c'est-à-dire l'ensemble des valeurs que peut prendre X_i

(3) $C = \{C_1, C_2, \ldots, C_k\}$ est l'ensemble des **contraintes**. Chaque contrainte C_j est une relation entre certaines variables de X, restreignant les valeurs que peuvent prendre simultanément ces variables.

Exemple : (1) $X = \{A, B, C, D\}$
(2) $D(A) = D(B) = D(C) = D(D) = \{0, 1\}$ ou $\{A, B, C, D\} \in 0, 1$
(3) $C = \{A \neq B\ ,\ C \neq D\ ,\ A + C < B\}$

Problème de satisfaction de contraintes

La résolution d'un CSP (X, D, C) consiste à **affecter des valeurs aux variables de telle sorte que toutes les contraintes soient satisfaites.**

Affectation (ou substitution)

Fait d'instancier des variables par des valeurs (prises dans les domaines des var.)

Une affectation est dite **totale** si elle instancie toutes les variables du problème.
Une affectation est dite **partielle** si elle n'en instancie qu'une partie.

Une affectation A **viole une contrainte** C_k si toutes les variables de C_k sont instanciées dans A et si la relation définie par C_k n'est pas vérifiées pour les valeurs des variables de C_k définies dans A

Exemples :

- $A_1 = \{(B, 0), (C, 1)\}$ est une affectation partielle qui instancie B à 0 et C à 1
- $A_2 = \{(A, 0), (B, 0)\}$ est une affectation partielle qui viole la contrainte $A \neq B$ (mais ne viole pas les deux autres contraintes)

Problème de satisfaction de contraintes

Affectation consistante et inconsistante

Une affectation (totale ou partielle) est :
- **consistante** si elle ne viole aucune contrainte
- **inconsistante** si elle viole une ou plusieurs contraintes

Solution

Une solution est une **affectation totale consistante**,
c'est-à-dire une valuation de toutes les variables du problème qui ne viole aucune contrainte

Exemples :

- $A_3 = \{(C, 0), (D, 1)\}$ est une affectation partielle consistante
- $A_2 = \{(A, 0), (B, 0)\}$ est une affectation partielle inconsistante
- $A_4 = \{(A, 0), (B, 1), (C, 0), (D, 1)\}$ est une affectation totale consistante : c'est une solution

Problème de satisfaction de contraintes

Ce que vous ne verrez pas aujourd'hui... (RDV en Master)

- Vérifier qu'une contrainte est "satisfaisable" est difficile !
- La difficulté du problème dépend :
  - du type des contraintes (linéaire ou non)
  - du domaine des variables (entiers, réels, ...)
  - etc.

Exercice classique : la coloration de cartes

On considère le pb de la coloration de la carte suivante en trois couleurs (rouge, vert et bleu). On désire affecter une couleur à chaque région tel que les régions adjacentes ne soient pas coloriées de la même couleur.

Exercice :

- Définir le CSP modélisant le problème ci-dessus.
- Donner une solution du problème

Problème de satisfaction de contraintes

Un autre exemple classique : les cryptarithmes

On souhaite résoudre le cryptarithme suivant :

$$\begin{array}{cccc} & T & W & O \\ + & T & W & O \\ \hline F & O & U & R \end{array}$$

Exercice : Définir le CSP modélisant ce problème.

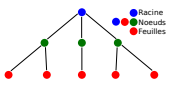
Recherche de solutions

Arbres de recherche

Arbre de recherche

Un **arbre de recherche** est un arbre dont les nœuds sont étiquetés par des **affectations (partielles)** du problème.

- La **racine** contient l'**affectation vide**
- Les **feuilles** contiennent des **affectations totales**



États (nœuds)

Un état contient les valeurs assignées jusqu'à maintenant

- **État inital** (racine) : Affectation vide
- **Fonction successeurs** : Étant donné un nœud/état, création d'un nouvel état en affectant une valeur à une variable non affectée sans créer de conflit
- **Test de but** (feuilles) : Teste si l'affectation totale est une solution

Exploration avec retour arrière

Même formultation de l'arbre de recherche pour tous les CSP :

- À chaque **nœud** : affectation d'**une seule** variable (var = valeur)
- Il y a $|\text{valeurs}|^{|\text{variables}|}$ feuilles
- L'assignation des variables est **commutative**
  ex : (WA=rouge suivi de NT=vert) $\Leftrightarrow$ (NT=vert suivi de WA=rouge)
- Toutes les solutions à n variables apparaissent à la profondeur n de l'arbre de recherche $\Longrightarrow$ "**Recherche en profondeur d'abord**"

Exploration avec retour arrière (algo sur le slide suivant)

- La recherche en profondeur d'abord pour les CSP avec l'assignation d'une seule variable à chaque tour est appelée **exploration avec retour arrière**
- Le retour arrière s'effectue lorsqu'il n'y a plus d'affectations
- Méthode très efficace qui peut résoudre de gros problème combinatoires (ex : le pb des n reines peut se résoudre avec $n = 25$)

Algorithme : Exploration avec retour arrière

Objectif : Trouver une affectation totale et consistante dans l'arbre

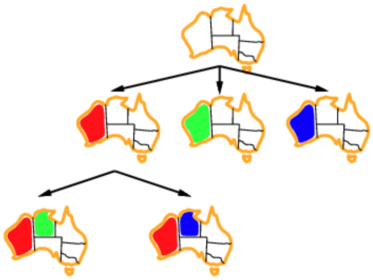
```
1:  affectation_courante ← affectation vide
2:  tant que l'arbre n'a pas été entièrement exploré faire
3:    tant que affectation_courante n'est pas totale faire
4:      Var ← choisir une variable non affectée
5:      pour tout Val ∈ domaine de Var (qui n'a pas encore été choisie) faire
6:        Val ← une valeur dans le domaine de Var (qui n'a pas encore été choisie)
7:        si Val est consistante (avec les contraintes) alors
8:          affectation_courante ← affectation_courante "+" Var = Val
9:          on sort du "pour tout" ⇔ On va à la ligne 11
10:       fin si
11:     fin pour
12:   fin tant que
13:   si affectation_courante est consistante (et totale) alors
14:     renvoyer affectation_courante (qui est donc une solution)
15:   sinon
16:     affectation_courante ← affectation_courante "-" Var = Val
17:   fin si
18: fin tant que
19: renvoyer qu'il n'y a pas d'affectation totale et consistante
```

Rq : Il n'y a pas besoin de construire l'arbre pour l'explorer...

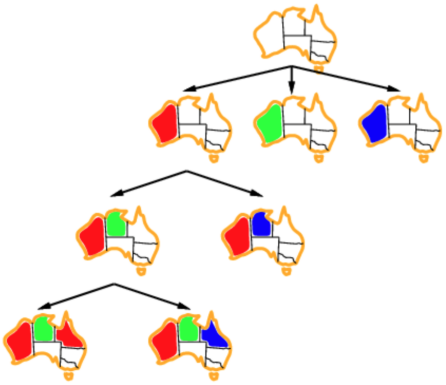
Exemple d'exécution de l'algorithme

Exemple d'exécution de l'algorithme

Exemple d'exécution de l'algorithme



Exemple d'exécution de l'algorithme



Comment améliorer l'efficacité de l'algorithme ?

- Quelle variable devrait être assignée au prochain niveau ?
- Dans quel ordre les valeurs devraient être essayées ?
- Quelles sont les implications des affectations de la variable courante sur les autres variables affectées ?
- Lorsqu'un chemin échoue, est-ce que l'exploration pourrait éviter de reproduire cet échec dans les chemins suivants ?

Ordre des variables

Comment choisir les variables ?
↪ via une **Heuristique** : algo permettant de trouver une "bonne" solution à un "prix" acceptable (comme le temps)

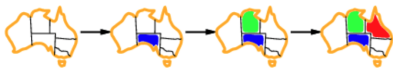
Exemples

- Heuristique du nombre minimum de valeurs restantes

⇒ Choisir la variable ayant le moins de valeurs possibles

- Heuristique du degré

⇒ Choisir la variable présente dans le plus de contraintes

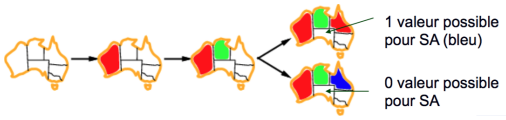


Ordre des valeurs

Comment choisir les valeurs ?
↪ En choisissant la moins contraignante

Exemple

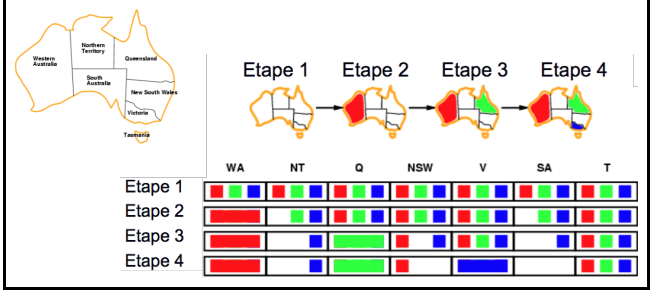
Choisir la valeur qui élimine le moins de choix pour les variables voisines



Forward checking : Implication des affectations

Le **Forward checking** permet de maintenir, en permanence, toutes les valeurs possibles pour chacune des variables

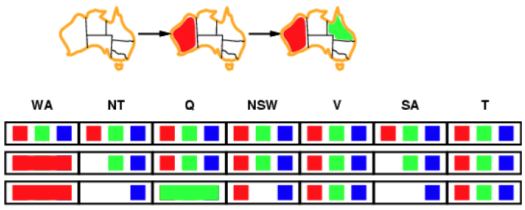
Exemple



Forward checking et Propagation de contraintes

Le Forward checking ne permet pas de détecter tous les problèmes
↪ La propagation de contraintes renforce rapidement les contraintes locales !

Exemple



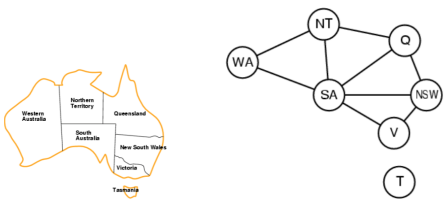
⇒ NT et SA ne peuvent pas tous les deux être bleus !

Représentation des contraintes

Si toutes les contraintes du CSP sont reliées à 2 variables
Alors les contraintes peuvent être représentées sous la forme d'un graphe

Certains algo de résolution de CSP utilisent les graphes pour accélérer la recherche

Exemple



Tasmanie est un sous problème indépendant

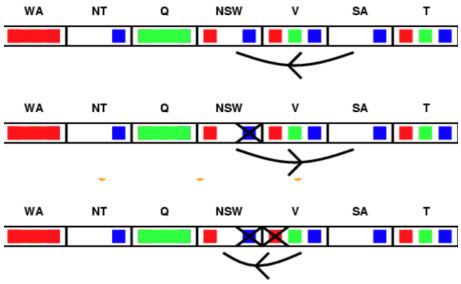
Propagation à l'aide de la cohérence des arcs

Cohérence des arcs

Un arc entre X et Y est cohérent si pour toutes les valeurs x de X, il y a au moins une valeur possible de y pour Y

Forme la plus simple de propagation

Faire en sorte que chaque arc soit cohérent pour trouver des branches de l'arbre menant à un échec plus vite que le forward checking
But : réduire le domaine des variables

Exemple de propagration via la cohérence des arcs

Si X perd une valeur, les voisins de X doivent être revérifiés